DRAGON

Divergence-Regulation for Adaptive Godunov-Oriented Numerics

Literary Overview & Reference Entries (LORE) Document

Bobbie Markwick

19 August 2026

Contents

Chapter 1

Physical Model & Numerical Methods

1.1 Governing Equations

The ideal MHD equations solved by DRAGON are, in conservative form:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) = 0 \tag{1.1a}$$

$$\frac{\partial}{\partial t}(\rho \mathbf{v}) + \nabla \cdot \left[\rho \mathbf{v}\mathbf{v} - \frac{1}{4\pi}\mathbf{B}\mathbf{B} + \left(p + \frac{\mathbf{B}^2}{8\pi}\right)\mathbb{I}\right] = 0 \tag{1.1b}$$

$$\frac{\partial E}{\partial t} + \nabla \cdot \left[\left(E + p + \frac{\mathbf{B}^2}{8\pi}\right)\mathbf{v} - (\mathbf{v}\cdot\mathbf{B})\mathbf{B}\right] = 0 \tag{1.1c}$$

$$\frac{\partial \mathbf{B}}{\partial t} = \nabla \times (\mathbf{v} \times \mathbf{B}) \tag{1.1d}$$

subject to the constraint

$$\nabla \cdot \mathbf{B} = 0. \tag{1.1e}$$

Here ρ is mass density, $\mathbf{v}$ the fluid velocity, $\mathbf{B}$ the magnetic field, and p the thermal pressure. E the total energy density

$$E = \frac{p}{\gamma - 1} + \frac{1}{2}\rho \mathbf{v}^2 + \frac{\mathbf{B}^2}{8\pi}, \tag{1.2}$$

where γ adiabatic index. DRAGON adopts Gaussian (non-rationalized) electromagnetic units, so the magnetic pressure and Lorentz force terms carry an explicit factor of 4π.

1.2 Initial Conditions

In DRAGON, problem initialisation consists of three steps:

1. Hydrodynamic Initialisation
2. Magnetic Field Initialization
3. (Optional) Custom Additional Initialisation

In the first step, the user implements `Problem::initialFluidState(x,y,z)` which returns a primitive hydrodynamic state $w = (\rho, \mathbf{v}, p)$ at position (x, y, z). The cell $w_{i,j,k}$ will be initialised with the state computed by this function with input position $((i + \frac{1}{2})\,\mathrm{d}x, (j + \frac{1}{2})\,\mathrm{d}y, (k + \frac{1}{2})\,\mathrm{d}z)$. Any value of $\mathbf{B}$ provided in this step will be overridden in the following step, though there are cases where it is useful to compute a preliminary value of $\mathbf{B}$ at this stage.

In the second step, the user implements `Problem::initialMagneticPotential(x,y,z)` which returns vector potential $\mathbf{A}$ at position (x, y, z). From this function, $\mathbf{B} = \nabla \times \mathbf{A}$ is initialised as follows:

1. $\mathbf{A}$ is calculated at every edge on the grid.

$$A^x_{i,j-\frac{1}{2},k-\frac{1}{2}} = \mathbf{A}\left[\left(i + \frac{1}{2}\right)\mathrm{d}x, j\,\mathrm{d}y, k\,\mathrm{d}z\right]_x \tag{1.3a}$$

$$A^y_{i-\frac{1}{2},j,k-\frac{1}{2}} = \mathbf{A}\left[i\,\mathrm{d}x, \left(j + \frac{1}{2}\right)\mathrm{d}y, k\,\mathrm{d}z\right]_y \tag{1.3b}$$

$$A^z_{i-\frac{1}{2},j-\frac{1}{2},k} = \mathbf{A}\left[i\,\mathrm{d}x, j\,\mathrm{d}y, \left(k + \frac{1}{2}\right)\mathrm{d}z\right]_z \tag{1.3c}$$

2. The normal component of $\mathbf{B}$ is calculated on each of the faces

$$B^x_{i-\frac{1}{2},j,k} = \frac{1}{4}\left(A^y_{i-\frac{1}{2},j,k-\frac{1}{2}} + A^z_{i-\frac{1}{2},j+\frac{1}{2},k} - A^y_{i-\frac{1}{2},j,k+\frac{1}{2}} - A^z_{i-\frac{1}{2},j-\frac{1}{2},k}\right) \tag{1.4a}$$

$$B^y_{i,j-\frac{1}{2},k} = \frac{1}{4}\left(A^z_{i-\frac{1}{2},j-\frac{1}{2},k} + A^x_{i,j-\frac{1}{2},k+\frac{1}{2}} - A^z_{i+\frac{1}{2},j-\frac{1}{2},k} - A^x_{i,j-\frac{1}{2},k-\frac{1}{2}}\right) \tag{1.4b}$$

$$B^z_{i,j,k-\frac{1}{2}} = \frac{1}{4}\left(A^x_{i,j-\frac{1}{2},k-\frac{1}{2}} + A^y_{i+\frac{1}{2},j,k-\frac{1}{2}} - A^x_{i,j+\frac{1}{2},k-\frac{1}{2}} - A^y_{i-\frac{1}{2},j,k-\frac{1}{2}}\right) \tag{1.4c}$$

3. $\mathbf{B}_{i,j,k}$ is calculated as an average of the adjacent faces

$$B^x_{i,j,k} = \frac{1}{2}\left(B^x_{i-\frac{1}{2},j,k} + B^x_{i+\frac{1}{2},j,k}\right) \tag{1.5a}$$

$$B^y_{i,j,k} = \frac{1}{2}\left(B^y_{i,j-\frac{1}{2},k} + B^y_{i,j+\frac{1}{2},k}\right) \tag{1.5b}$$

$$B^z_{i,j,k} = \frac{1}{2}\left(B^z_{i,j,k-\frac{1}{2}} + B^z_{i,j,k+\frac{1}{2}}\right) \tag{1.5c}$$

This procedure ensures that $\mathbf{B}$ is initialised with zero divergence everywhere.

In the final step, the user implements `Problem::completeProblemInit(grid)` to perform any additional initialisation steps on `grid`.

1.3 Boundary Conditions

DRAGON supports several types of boundary conditions, and provides capability for defining customisable boundary conditions. All boundary conditions are subclasses of the `GhostFill` subclass, which are responsible for filling in the ghost cells on a `Grid` object. To set the boundary conditions for a problem, set the `boundary` property of the `Grid` class, for example

```
grid.boundary = Periodic();
```

Every time the grid advances, it will call the `apply(Grid&)` function on its boundary object.

A boundary object can be restricted to specific faces by passing a string argument to the constructor. In this string, specify any combination of X, Y, Z, optionally adding plus or minus after any or all of these to only include the respective side[1]. For example, `Reflective("XZ-")` uses reflective boundary conditions on the X_-, X_+, and Z_- boundaries. Any faces not explicitly specified will default to outflow.

Boundary conditions can be combined using the `+` operator. For example:

```
grid.boundary = Reflective("XZ-") + Periodic("Y");
```

defines a box which is reflective in the $X_\pm$ and Z_- boundaries, periodic in the Y direction, and defaults to outflow on the Z_+ boundary. Boundaries are applied in the order they are specified, with any implicit outflow being applied first. In the example, the reflective boundaries are applied before the periodic boundaries.

In addition to passing a face-restriction string, the GhostFill constructor also takes a boolean `corner_ghosts`. This value is true by default, and you should normally leave it that way. If `corner_ghosts` is true, then all ghosts on the corresponding side of the boundary will be filled. If it is false, then only the ghosts which are out of bounds in a single dimension are filled, and ghosts in the corner regions are assumed to have already been properly filled by a previous boundary object.

DRAGON includes the following built in boundary classes

1.3.1 Outflow

The `Boundary::Outflow` class implements outflow (Neumann) boundary conditions, filing each ghost with the state of the nearest physical cell. Outflow boundaries are the default boundary if none is set. Negative-side and positive-side ghosts on an X boundary are filled as

$$\begin{pmatrix} \rho_{(-g,j,k)} \\ \mathbf{v}_{(-g,j,k)} \\ p_{(-g,j,k)} \\ B^y_{(-g,j,k)} \\ B^z_{(-g,j,k)} \end{pmatrix} = \begin{pmatrix} \rho_{(0,j,k)} \\ \mathbf{v}_{(0,j,k)} \\ p_{(0,j,k)} \\ B^y_{(g-1,j,k)} \\ B^z_{(g-1,j,k)} \end{pmatrix}, \qquad \begin{pmatrix} \rho_{(n_x-1+g,j,k)} \\ \mathbf{v}_{(n_x-1+g,j,k)} \\ p_{(n_x-1+g,j,k)} \\ B^y_{(n_x-1+g,j,k)} \\ B^z_{(n_x-1+g,j,k)} \end{pmatrix} = \begin{pmatrix} \rho_{(n_x-1,j,k)} \\ \mathbf{v}_{(n_x-1,j,k)} \\ p_{(n_x-1,j,k)} \\ B^y_{(n_x-1,j,k)} \\ B^z_{(n_x-1,j,k)} \end{pmatrix} \tag{1.6}$$

Normal magnetic fields however are linearly extrapolated to preserve $\nabla \cdot \mathbf{B} = 0$:

$$B^x_{(-g,j,k)} = 2B^x_{(-g+1,j,k)} - B^x_{(-g+2,j,k)} \tag{1.7a}$$

$$B^x_{(n_x+g,j,k)} = 2B^x_{(n_x-1+g,j,k)} - B^x_{(n_x-2+g,j,k)} \tag{1.7b}$$

Calling `Boundary::Outflow::Gated` instead of the outflow constructor will return a modified outflow condition designed to inhibit accidental inflow. This variant follows all outflow criteria as normal, except that if the normal velocity is directed inwards to the physical region ($v_x > 0$ on an X_- boundary, $v_x < 0$ on an X_+ boundary), that component will be set to zero in the boundary region.

1.3.2 Periodic

The `Boundary::Periodic` class implements periodic boundary conditions, filling ghosts such that the ghost cells near the positive boundary match the physical cells near the corresponding negative boundary. Negative-side ghosts are filled as $w_{-g,j,k} = w_{n_x-g,j,k}$ and positive-side ghosts are filled as $w_{n_x+g,j,k} = w_{g,j,k}$. Face-magnetic ghosts are filled likewise for half-integer values of g.

[1] Plus and minus restriction is not supported for periodic boundary conditions

Note that periodic boundary conditions require that both the plus and minus boundaries of any given dimension are used; if restriction such as `Periodic("X-")` is attempted, the object will initialise such that sides will be used (in this case, equivalent to `Periodic("X")`).

1.3.3 Reflective

The `Boundary::Reflective` class implements reflective boundary conditions, filling ghosts such that they mirror the nearby physical cells. These conditions mimic a stationary wall, which in MHD is also taken to be perfectly conductive. Negative-side and positive-side ghosts on an X boundary are filled as

$$\begin{pmatrix} \rho_{(-g,j,k)} \\ v^x_{(-g,j,k)} \\ v^y_{(-g,j,k)} \\ v^z_{(-g,j,k)} \\ p_{(-g,j,k)} \\ B^x_{(-g,j,k)} \\ B^y_{(-g,j,k)} \\ B^z_{(-g,j,k)} \end{pmatrix} = \begin{pmatrix} \rho_{(g-1,j,k)} \\ -v^x_{(g-1,j,k)} \\ v^y_{(g-1,j,k)} \\ v^z_{(g-1,j,k)} \\ p_{(g-1,j,k)} \\ B^x_{(g-1,j,k)} \\ -B^y_{(g-1,j,k)} \\ -B^z_{(g-1,j,k)} \end{pmatrix}, \qquad \begin{pmatrix} \rho_{(n_x-1+g,j,k)} \\ v^x_{(n_x-1+g,j,k)} \\ v^y_{(n_x-1+g,j,k)} \\ v^z_{(n_x-1+g,j,k)} \\ p_{(n_x-1+g,j,k)} \\ B^x_{(n_x-1+g,j,k)} \\ B^y_{(n_x-1+g,j,k)} \\ B^z_{(n_x-1+g,j,k)} \end{pmatrix} = \begin{pmatrix} \rho_{(n_x-g,j,k)} \\ -v^x_{(n_x-g,j,k)} \\ v^y_{(n_x-g,j,k)} \\ v^z_{(n_x-g,j,k)} \\ p_{(n_x-g,j,k)} \\ B^x_{(n_x-g,j,k)} \\ -B^y_{(n_x-g,j,k)} \\ -B^z_{(n_x-g,j,k)} \end{pmatrix} \tag{1.8}$$

Face-magnetic ghosts are filled likewise for half-integer values of g.

1.3.4 Fixed

The `Boundary::Fixed` class implements fixed (Dirichlet) boundary conditions, filling ghosts with a constant uniform state. When constructing a fixed boundary, the primitive fluid state is the first argument.

Fixed boundaries are fine to use in hydrodynamic problems, but aren't recommended for non-trivial MHD problems since their interaction with constrained transport hasn't been tested yet.

1.3.5 Ignore

The `Boundary::Ignore` class is a no-op boundary. Generally you should not use this in your problems, but the it is used internally to support domain decomposition.

1.4 Multidimensional Integration Schemes

1.4.1 Operator-Split (Strang) Update

In split mode, each timestep is built from a sequence of 1D sweeps along coordinate directions, each solved with the same 1D Godunov kernel (`Godunov::sweep`): apply boundary conditions (Section 1.3), reconstruct interface states with MUSCL (Section 1.6), solve the Riemann problem (Section 1.7) at each face, and difference the resulting fluxes into every cell in a single pass down the array, reusing the right-going flux of one interface as the left-going flux of the next.

A multidmensional split sweep consists of sweeps in the x, y, and (if applicable) z directions. A sweep along y or z is performed by transposing (via `swappedXY` or `swappedXZ`) each 1D line of cells so the swept direction is treated as x, running the same kernel as the x sweep, and transposing back. This approach means that only one 1D solver needs to be implemented and tested. Each split step operates on a scratch copy of the grid so that, if any sub-sweep fails the physicality check, the original state is left untouched and the whole step can be retried at a smaller Δt.

To avoid the directional bias a fixed sweep order would introduce, DRAGON alternates or rotates the sweep order with every step. In two dimensions, this is done by alternating by between XYX and YXY, both with a $(\frac{\Delta t}{2}, \Delta t, \frac{\Delta t}{2})$ pattern. In three dimensions, this is done by rotating through six orderings: the cyclic permutations $(XYZYX)$, $(ZXYXZ)$, $(YZXZY)$, and the anticyclic permutations $(ZYXYZ)$, $(XZYZX)$, and $(YXZXY)$ all with a $(\frac{\Delta t}{2}, \frac{\Delta t}{2}, \Delta t, \frac{\Delta t}{2}, \frac{\Delta t}{2})$ pattern.

1.4.2 Unsplit (CTU) Update

The unsplit driver (`Grid::unsplit_step`) advances all directions simultaneously in a single conservative update, following this sequence:

1. **MUSCL**: compute preliminary half-step interface states in every direction (Section 1.6).
2. **CTU**: apply transverse corrections to those interface states (Section 1.6.4), assuming `CTU` is enabled in `Config.h`
3. **Final fluxes**: solve the Riemann problem at every face using the (possibly CTU-corrected) interface states (Section 1.7).
4. **Update hydrodynamic states**: apply the fluxes conservatively to a scratch copy.
5. **Constrained Transport**: extract, upwind, and integrate the electric field, then advance the face-noraml **B** and recompute body-centered fields (Section 1.8).
6. **Physicality check** – verify every updated cell is physical; if not, throw to trigger a step restart.
7. **Synchronization** – in a domain-decomposed run, wait for all other subgrids to reach the same checkpoint (and confirm none of them failed) before proceeding.
8. **Commit** – copy the scratch states computed by steps 4 and 5 back into the live grid.

For the flux application in step 4, `Godunov::applyFluxes` takes fluxes F^X, F^Y, F^Z through every face of a cell and updates the conservative state before converting back to primitive:

$$U^{n+1}_{i,j,k} = U^n_{i,j,k} + \frac{\Delta t}{\Delta x}\left(F^X_{i-\frac{1}{2},j,k} - F^X_{i+\frac{1}{2},j,k}\right) + \frac{\Delta t}{\Delta y}\left(F^Y_{i,j-\frac{1}{2},k} - F^Y_{i,j+\frac{1}{2},k}\right) + \frac{\Delta t}{\Delta z}\left(F^Z_{i,j,k-\frac{1}{2}} - F^Z_{i,j,k+\frac{1}{2}}\right), \tag{1.9}$$

throwing if any updated cell is non-finite (this is a coarser, cheaper check than the full physicality test applied after the CT update).

1.5 Time Integration

1.5.1 Per-Cell Signal Speed

For a single cell with primitive state w, DRAGON computes a local CFL signal speed from the fast magnetosonic speed c_f (sound speed c_s in hydrodynamic builds), combined across the active grid

directions in one of three ways, selected by `CFL_CALCULATION` in `Config.h`:

$$\texttt{CFL\_MAX:} \qquad s_{\max} = \max_{d\in\{x,y,z\}} \left(\frac{|v_d| + c}{\Delta d} \right), \tag{1.10}$$

$$\texttt{CFL\_ADD:} \qquad s_{\text{add}} = \sum_{d\in\{x,y,z\}} \frac{|v_d| + c}{\Delta d}, \tag{1.11}$$

$$\text{power } p: \qquad s_p = \left[\sum_{d\in\{x,y,z\}} \left(\frac{|v_d| + c}{\Delta d} \right)^p \right]^{1/p}, \tag{1.12}$$

where a direction d is only included if $\Delta d > 0$ (i.e. it is an active grid dimension). Equation 1.10 is typically appropriate for dimensional splitting, while the more conservative equation 1.11 is appropriate for unsplit updates where waves can cross diagonally in a single step. Equation 1.12 interpolates between the two as p is tuned, approaching equation Equation (1.10) as $p \to \infty$.

1.5.2 Grid-Wide Timestep

The global timestep bound is derived from the inverse of the fastest signal speed for any one cell in the physical region of the grid:

$$\Delta t \le C_{\text{CFL}} \min_{i,j,k} \left(\frac{1}{s_{i,j,k}} \right) \tag{1.13}$$

where $s_{i,j,k}$ are the speeds calculated in the previous section for each individual cell and $C_{\text{CFL}} \le 1$ is the coefficient given by `CONFIG::CFL_coeff`. The first layer of ghosts are also included in the calculation, but additional ghosts are ignored as they are only used to correct the states seen at the interfaces. If the resulting timestep is nonpositive, `CFL::cfl_time` throws an exception, since no finite timestep can be justified in that case.

The `Grid::advance(dt)` function is responsible for controlling the time advancement as follows

1. Each step begins with the application of boundary conditions.
2. Timestep Δt_1 is taken to be the lesser of `dt` and the bound given by 1.13.
3. Advance the problem by time Δt_1. If an exception is thrown, halve Δt_1 and try again.
4. Subtract Δt_1 from `dt`.
5. Repeat from step 1 until `dt` falls below `CONFIG::Timestep_Tolerance` (10^{-14} by default)

This retry-with-halving behaviour is what lets the physicality checks recover from an occasional bad step (e.g. from an overly aggressive CFL coefficient) without aborting the whole simulation, at the cost of re-doing the failed substep at a smaller Δt.

1.6 Reconstruction

DRAGON reconstructs left/right interface states using the MUSCL (Monotonic Upstream-centered Scheme for Conservation Laws) approach of van Leer, applied to the primitive variables $w = (\rho, \mathbf{v}, p, \mathbf{B})$.

1.6.1 Piecewise-Linear Slopes

For each cell i, a limited slope Δ_i is computed from the neighbouring cell averages,

$$\Delta_i = \text{limiter}(w_i - w_{i-1},\ w_{i+1} - w_i)\,, \tag{1.14}$$

where w_i are the primitive variables in the ith cell, and limiter is the limiter function selected by setting `MUSCL_DEFAULT_LIMITER` in `Config.h`. The following limiter functions are implemented:

$$\text{minmod}(a,b) = \begin{cases} a & |a| < |b| \text{ and } ab > 0, \\ b & |b| \le |a| \text{ and } ab > 0, \\ 0 & ab \le 0. \end{cases} \tag{1.15a}$$

$$\text{MC}(a,b) = \begin{cases} 2a & |a| \le \left\{|b|, \frac{1}{4}|a+b|\right\} \text{ and } ab > 0, \\ 2b & |b| \le \left\{|a|, \frac{1}{4}|a+b|\right\} \text{ and } ab > 0, \\ \frac{1}{2}|a+b| & \frac{1}{4}|a+b| < \{|a|, |b|\} \text{ and } ab > 0, \\ 0 & ab \le 0. \end{cases} \tag{1.15b}$$

$$\text{superbee}(a,b) = \begin{cases} 2a & 2|a| \le |b| \text{ and } ab > 0, \\ 2b & 2|b| \le |a| \text{ and } ab > 0, \\ a & |b| < |a| < 2|b| \text{ and } |a| > |b| \text{ and } ab > 0, \\ b & |a| \le |b| < 2|a| \text{ and } |b| > |a| \text{ and } ab > 0, \\ 0 & ab \le 0. \end{cases} \tag{1.15c}$$

$$\text{vanLeer}(a,b) = \begin{cases} \frac{2ab}{a+b} & ab > 0, \\ 0 & ab \le 0. \end{cases} \tag{1.15d}$$

$$\text{vanAlbada}(a,b) = \begin{cases} \frac{ab(a+b)}{a^2+b^2} & ab > 0, \\ 0 & ab \le 0. \end{cases} \tag{1.15e}$$

The MC limiter is selected by default, as it provides a reasonable balance between robustness and nondiffusiveness.

Once Δ_i is computed, the piecewise-linear interface states are computed as

$$w^L_{i+1/2} = w_i + \tfrac{1}{2}\Delta_i, \qquad w^R_{i+1/2} = w_{i+1} - \tfrac{1}{2}\Delta_{i+1}. \tag{1.16}$$

Note that in code $w^L_{i+1/2}$ is generally referred to as `_xR[i]` and $w^R_{i+1/2}$ as `_xL[i+1]`, denoting them relative to the cells from which they originate rather than the Riemann problem to which they will be passed.

1.6.2 Half-Step Time Predictor

To achieve second-order accuracy in time, the interface states are then advanced by a half time step using the flux difference across the cell,

$$w^{L,n+1/2}_{i+1/2} = w^L_{i+1/2} + \frac{\Delta t}{2\Delta x}\left[F(w^R_{i-1/2}) - F(w^L_{i+1/2})\right] + \frac{\Delta t}{2} S_{\text{MHD}}, \tag{1.17a}$$

$$w^{R,n+1/2}_{i-1/2} = w^R_{i-1/2} + \frac{\Delta t}{2\Delta x}\left[F(w^R_{i-1/2}) - F(w^L_{i+1/2})\right] + \frac{\Delta t}{2} S_{\text{MHD}} \tag{1.17b}$$

where S_{MHD} are MHD source terms (see section 1.6.3). Note that $w^{LR}_{i+1/2}$ are converted from primitive to conservative variables before the above operation, and then converted back to primitive variables when assigned to $w^{LR,n+1/2}_{i+1/2}$. DRAGON's fluid arithmetic allows this to be done implicitly, though MUSCL does the primitive-to-conservative conversion explicitly to avoid redundant conversion.

If for a given cell the above procedure produces a nonphysical result for $w^R_{i-1/2}$ or $w^L_{i+1/2}$, then both of these interface states fall back to the first-order reconstruction $w^R_{i-1/2} = w^L_{i+1/2} = w_i$. The above procedure can also be disabled (in which case first-order reconstruction is always used) by disabling `MUSCL_Hancock` in `Config.h`, though this isn't recommended.

1.6.3 MHD Source terms

When reconstructing MHD states, additional source terms S_{MHD} are needed to ensure the reconstructed state remains consistent with the solenoid constraint. These terms are computed from the face-normal fields $B^x_{i+1/2,j,k}$ (see section 1.8) and their normal derivatives

$$\frac{\partial B_x}{\partial x} = \frac{1}{\Delta x}\left(B^x_{i+1/2,j,k} - B^x_{i-1/2,j,k}\right) \tag{1.18a}$$

$$\frac{\partial B_y}{\partial x} = \frac{1}{\Delta y}\left(B^y_{i,j+1/2,k} - B^y_{i,j-1/2,k}\right) \tag{1.18b}$$

$$\frac{\partial B_z}{\partial x} = \frac{1}{\Delta z}\left(B^z_{i,j,k+1/2} - B^z_{i,j,k-1/2}\right) \tag{1.18c}$$

Source terms are also not needed for density and momentum, as these terms do not interact with magnetic field outside of their fluxes. For the transverse magnetic field, the source terms are given as[2]

$$S_{\text{MHD}}(B_y) = v_y\left[\text{minmod}\left(\frac{\partial B_x}{\partial x}, -\frac{\partial B_y}{\partial y}\right) - \frac{\Delta_i(B_x)}{\Delta x}\right] \tag{1.19}$$

and similarly for B_z. For the normal term however $S_{\text{MHD}}(B_x) = 0$, as $(B_x)^{L,n+1/2}_{i+1/2}$ and $(B_x)^{R,n+1/2}_{i+1/2}$ will be set to $B^x_{i+1/2}$ after MUSCL reconstruction is complete. The energy source term is then given by

$$S_{\text{MHD}}(E) = \frac{\mathbf{B}}{4\pi} \cdot S_{\text{MHD}}(\mathbf{B}) \tag{1.20}$$

where $\mathbf{B}$ are taken from w_i.

1.6.4 CTU Transverse Correction

The plain MUSCL half-states of Section 1.6 account only for the wave structure normal to each interface; used directly in an unsplit update, they under-resolve genuinely multidimensional flow. DRAGON corrects for this following the procedure given by Gardiner & Stone (2008, hereafter GS08).

[2]Note that minmod is used regardless of which limiter is used for equation 1.14.

Hydrodynamics. The correction (`Godunov::correctState`) is a half-step transverse flux difference applied to *both* sides of each interface state,

$$w_d^{L,\mathrm{corr}} = w_d^L - \frac{\Delta t}{2\Delta d'}\left[F_{d'}(w_{d'+\frac{1}{2}}) - F_{d'}(w_{d'-\frac{1}{2}})\right], \qquad w_d^{R,\mathrm{corr}} = w_d^R - \frac{\Delta t}{2\Delta d'}\left[\cdots\right], \tag{1.21}$$

for each transverse direction $d' \neq d$. In 3D, GS08's "12-solve" algorithm additionally corrects each transverse flux itself with a one-third-weighted contribution from the *third* direction before using it (`computeCTUFlux_X/Y/Z`), so that corner-crossing waves are not double-counted.

Magnetohydrodynamics. In MHD, DRAGON first computes a half-step-advanced $\mathbf{B}$ and body-centred $\mathbf{E}$ using preliminary, uncorrected fluxes (by solving the Riemann problem at the interfaces, see Section 1.7) and one CT update (see section 1.8) at $\Delta t/2$. From here, the conservative hydrodynamic variables $(\rho, \rho\mathbf{v}, E)$ are updated similarly as to the hydrodynamic case, except that corrections from both transverse dimensions are applied simultaneously instead of feeding one update into the other. Instead of fluxes, magnetic fields receive their corrections from the electric fields $\mathbf{E}$ computed in the preliminary CT update. For the x interface, this is given by:

$$B_{y,i\pm\frac{1}{2},j,k}^{LR,n+\frac{1}{2}} = B_{y,i\pm\frac{1}{2},j,k}^{LR,n} + \frac{\Delta t}{4\Delta z}\left[\mathbf{E}^x_{i,j-\frac{1}{2},k-\frac{1}{2}} - \mathbf{E}^x_{i,j-\frac{1}{2},k+\frac{1}{2}} + \mathbf{E}^x_{i,j+\frac{1}{2},k-\frac{1}{2}} - \mathbf{E}^x_{i,j+\frac{1}{2},k+\frac{1}{2}}\right] + SB_y \tag{1.22a}$$

$$B_{z,i\pm\frac{1}{2},j,k}^{LR,n+\frac{1}{2}} = B_{z,i\pm\frac{1}{2},j,k}^{LR,n} + \frac{\Delta t}{4\Delta y}\left[\mathbf{E}^x_{i,j+\frac{1}{2},k-\frac{1}{2}} - \mathbf{E}^x_{i,j-\frac{1}{2},k-\frac{1}{2}} + \mathbf{E}^x_{i,j+\frac{1}{2},k+\frac{1}{2}} - \mathbf{E}^x_{i,j-\frac{1}{2},k+\frac{1}{2}}\right] + SB_z \tag{1.22b}$$

and similarly for the y and z interfaces. The normal components of $\mathbf{B}$ meanwhile are set to equal to the face-normal values from the staggered grid.

However, the MHD correction additionally needs to keep the corrected interface states consistent with the CT electric field, following GS08 eqs. 51–59. These additional[3] corrections are

$$SB_y = \frac{\Delta t}{2} v_y \mathrm{minmod}\left(\frac{\partial B_x}{\partial x}, -\frac{\partial B_z}{\partial z}\right), \qquad SB_z = \frac{\Delta t}{2} v_z \mathrm{minmod}\left(\frac{\partial B_x}{\partial x}, -\frac{\partial B_y}{\partial y}\right) \tag{1.23a}$$

$$SE = \frac{\Delta t}{8\pi}\left[B_y v_y \mathrm{minmod}\left(\frac{\partial B_x}{\partial x}, -\frac{\partial B_z}{\partial z}\right) + B_z v_z \mathrm{minmod}\left(\frac{\partial B_x}{\partial x}, -\frac{\partial B_y}{\partial y}\right)\right] \tag{1.23b}$$

for x-interfaces and similarly for y and z interfaces. The derivatives $\frac{\partial B_i}{\partial x_i}$ are the same as computed in section 1.6.3. Note that GS08 also includes a momentum term, but our MUSCL implementation renders this term unnecessary.

1.7 Riemann Solvers

The Riemann problem takes as inputs the left and right states w_L and w_R. In the context of a Godunov code, the relevant solution is the flux through the $x = 0$ plane. In DRAGON, this can be obtained by calling `Riemann(wL,wR).flux()`. This function uses whichever Riemann solver is selected by `RIEMANN_DEFAULT` in `Config.h`.

In multidimensional problems, the Riemann problem will also need to be solved on the y and z interfaces. `Riemann(wL,wR).flux_Y()` and `Riemann(wL,wR).flux_Z()` accomplish this by swapping x and y (or x and z), solving the problem as if it were an x interface, and then swapping the output back. `Riemann(wL,wR).flux_X()` is also provided for code symmetry, though this simply calls `Riemann(wL,wR).flux()` as no swapping is necessary.

[3] That is, these terms are in addition to, not in place of equations 1.22 and the corresponding hydrodynamic update

1.7.1 Exact (Hydrodynamics Only)

The Exact Riemann solver provides an exact solution to the Hydrodynamic Riemann problem, but as it generally requires an iterative procedure it is more costly than the other solvers. This solver can be called directly as `Riemann(wL,wR).exact().flux()`[4]

The exact solver iterates on the velocity-jump function

$$f(p,w)=\begin{cases}(p-p_w)\sqrt{\dfrac{2/[(\gamma+1)\rho_w]}{p+\frac{\gamma-1}{\gamma+1}p_w}} & p>p_w \text{ (shock)},\\ \dfrac{2c_{s,w}}{\gamma-1}\left[\left(\dfrac{p}{p_w}\right)^{\frac{\gamma-1}{2\gamma}}-1\right] & p\le p_w \text{ (rarefaction)},\end{cases} \tag{1.24}$$

to solve $f(p^*,w_L)+f(p^*,w_R)+(v_{x,R}-v_{x,L})=0$ for p^*.

When `Exact_Rarefactions_Check` is enabled in `Config.h`, DRAGON first tests whether the wave pattern will be two rarefactions ($f(p_{\min},w_L)+f(p_{\min},w_R)+v_{x,R}-v_{x,L}\ge 0$ with $p_{\min}=\min(p_L,p_R)$); if so, it uses the closed-form Two-Rarefaction Riemann Solver (TRRS) instead of iterating. Otherwise DRAGON starts from the initial guess $p_0^*=(p_L+p_R)/2$ and uses a damped Newton step ($\delta p^*=\min(f/f',\,0.8\,p^*)$) to prevent divergence to negative pressure. The iteration terminates when the relative change in p^* falls below `CONFIG::ExactRiemann_Tolerance` or `CONFIG::ExactRiemann_MaxIters` is exhausted.

Once p^* is calculated, star-region velocity and densities are given by

$$v_x^*=\frac{1}{2}\left[v_{x,L}+v_{x,R}+f(p^*,w_R)-f(p^*,w_L)\right], \tag{1.25a}$$

$$\rho_K^*=\begin{cases}\rho_K\left(\dfrac{p^*+\frac{\gamma-1}{\gamma+1}p_K}{\frac{\gamma-1}{\gamma+1}p^*+p_K}\right) & p^*>p_K \text{ (shock)},\\ \rho_K\left(p^*/p_K\right)^{1/\gamma} & p^*\le p_K \text{ (rarefaction)},\end{cases} \qquad K\in\{L,R\}. \tag{1.25b}$$

From here, the solution is sampled along the requested ray x/t (outside the wave fan, inside a shock's jump, or inside a rarefaction fan) to produce the flux; the sampling routine handles the left side of the contact by mirroring the problem (swapping $L\leftrightarrow R$ and negating normal velocity) and solving as if it were the right side to ensure implementation symmetry.

1.7.2 HLL

The two-wave Harten–Lax–van Leer solver approximates the star region as a single uniform state bounded by a pair of waves with speeds S_L and S_R. This solver can be called as `Riemann(wL,wR).HLL()`, or if wave speed estimates are already known, `Riemann(wL,wR).HLL(SL,SR)`.

Normal Magnetic Fields

In MHD, it is important that both sides of the problem have the same normal component of B. For HLL and all other solvers that support MHD, this is done by setting both to $\frac{1}{2}\left(B_x^L+B_x^R\right)$.

[4]The intermediate object given by `Riemann(wL,wR).exact()` contains the star states as well as the initial problem; calling `flux(x_t)` on this object then computes the flux through the trajectory $x/t=$`x_t`, with `x_t=0` as the default.

Roe-Averaged Intermediate State

HLL and HLLC both use a shared Roe-averaged state M (`HLL::roeAvg`) to sharpen their outer wave-speed estimates:

$$\begin{pmatrix} \rho_M \\ \mathbf{v}_M \\ \mathbf{B}_M \\ H_M \\ p_M \end{pmatrix} = \begin{pmatrix} \sqrt{\rho_L \rho_R} \\ \dfrac{\sqrt{\rho_L}\,\mathbf{v}_L + \sqrt{\rho_R}\,\mathbf{v}_R}{\sqrt{\rho_L} + \sqrt{\rho_R}} \\ \dfrac{\sqrt{\rho_L}\,\mathbf{B}_L + \sqrt{\rho_R}\,\mathbf{B}_R}{\sqrt{\rho_L} + \sqrt{\rho_R}} \\ \dfrac{\sqrt{\rho_L}\,H_L + \sqrt{\rho_R}\,H_R}{\sqrt{\rho_L} + \sqrt{\rho_R}} \\ (1 - \frac{1}{\gamma})\left(\rho_M H_M - \frac{1}{2}\rho_M \mathbf{v}_M^2\right) \end{pmatrix} \tag{1.26}$$

where $H_K = \frac{1}{\rho}\left(E_K + p_K + \frac{\mathbf{B}_K^2}{8\pi}\right)$ is specific enthalpy.

HLL Wave Estimates

Outer wave speeds combine the physical-state and Roe-averaged signal speeds,

$$S_L = \min\left(v_{x,L} - c_L,\ v_{x,M} - c_M\right), \qquad S_R = \max\left(v_{x,R} + c_R,\ v_{x,M} + c_M\right), \tag{1.27}$$

where c is the fast magnetosonic speed (sound speed in pure hydrodynamics) evaluated at L, R, and the Roe average M.

HLL Flux

Once the above speed estimates are calculated, the flux is then found via `Riemann(wL,wR).HLL(SL,SR)`[5]

The standard two-state flux is given by (Harten, Lax, & van Leer 1983)

$$F_{\text{HLL}}(S_L, S_R) = \begin{cases} F_L & S_L \geq 0, \\ \frac{S_R F_L - S_L F_R + S_L S_R (U_R - U_L)}{S_R - S_L} & S_L < 0 < S_R, \\ F_R & S_R \leq 0, \end{cases} \tag{1.28}$$

where F_L and F_R are the fluxes computed directly from conservative states U_L and U_R respectively.

1.7.3 HLLE

The HLLE solver implemented in `Riemann::HLLE()` uses the same two-state flux formula as HLL, but with the Einfeldt (1988) wave-speed bounds to additionally guarantee positivity of the intermediate density.

These speeds are given as

$$S_L = \min(v_{x,L} - c_L,\ v_{x,M} - d), \qquad S_R = \max(v_{x,R} + c_R,\ v_{x,M} + d), \tag{1.29}$$

where

$$d = \sqrt{\bar{a}^2 + \tfrac{1}{2}\sqrt{\rho_L \rho_R}\bar{v}_x^2}, \qquad \bar{a^2} = \frac{\sqrt{\rho_L}c_L^2 + \sqrt{\rho_R}c_R^2}{\sqrt{\rho_L} + \sqrt{\rho_R}}, \qquad \bar{v}_x = \frac{v_{x,R} - v_{x,L}}{\sqrt{\rho_L} + \sqrt{\rho_R}}. \tag{1.30}$$

[5] If you have your own method for calculating the wave speed estimates, you can call this method yourself instead of using the existing HLL machinery.

1.7.4 HLLC (Hydrodynamics Only)

HLLC (Toro, Spruce, & Speares 1994) restores the contact discontinuity smeared out by HLL/HLLE.

`Riemann(wL,wR).HLLC()` uses the same outer wave speeds given in section 1.7.2, or `Riemann(wL,wR).HLLC(SL,SR)` can be called directly given alternative estimates. The speed of the restored contact wave is then given by

$$S_M = \frac{p_R^* - p_L^*}{\rho_R(v_{x,R} - S_R) - \rho_L(v_{x,L} - S_L)} \tag{1.31}$$

where $p_K^* = p_K + \rho_K v_{x,K}(v_{x,K} - S_K)$.

Using this speed, identify the relevant side $X \in \{L, R\}$[6] and outer speed S_X . From there, the star-region conservative state is

$$U^* = U_X \left(\frac{S_X - v_{x,X}}{S_X - S_M} \right), \tag{1.32}$$

except that $(\rho v_x)^* = \rho^* S_M$ and E^* is given by

$$E^* = E_K \left(\frac{S_X - v_{x,X}}{S_X - S_M} \right) + p_X \left(\frac{S_M - v_{x,X}}{S_X - S_M} \right) + \rho^*(S_M - v_{x,X})S_M \tag{1.33}$$

From here, the final star-state flux is given by

$$F = F_X + S_X(U^* - U_X). \tag{1.34}$$

1.7.5 HLLD (MHD Only)

HLLD (Miyoshi & Kusano 2005) extends the contact-resolving idea of HLLC to MHD by resolving five waves: the two outer fast waves (S_L, S_R), two rotational/Alfvén waves (S_L^*, S_R^*), and the contact wave (S_M).

Wave speeds

Unlike other HLL solvers which use rho-averaging, HLLD's outer speeds use the fast magnetosonic speed c_f directly,

$$S_L = \min(v_{x,L} - c_{f,L},\ v_{x,R} - c_{f,R}), \qquad S_R = \max(v_{x,L} + c_{f,L},\ v_{x,R} + c_{f,R}). \tag{1.35}$$

If $S_L \geq 0$ or $S_R \leq 0$, then as with other HLL solvers the flux is simply F_L or F_R.

The contact wave speed is given by

$$S_M = \frac{(\varrho_R v_{x,R} - \varrho_L v_{x,L}) - (p_{T,R} - p_{T,L})}{\varrho_R - \varrho_L} \tag{1.36}$$

where $p_{T,K} = p_K + \frac{1}{8\pi}B_K^2$ is the total pressure and $\varrho_K = \rho_K(S_K - v_{x,K})$ are the mass fluxes. Alfvén speeds and star densities are then

$$S_L^* = S_M - \frac{|B_x|}{\sqrt{4\pi\rho_L^*}}, \qquad S_R^* = S_M + \frac{|B_x|}{\sqrt{4\pi\rho_R^*}}, \qquad \rho_K^* = \frac{\varrho_K}{S_K - S_M}. \tag{1.37}$$

[6] $X = L$ if $S_M > 0$, otherwise $X = R$

Outer Star States

Given total pressures $p_T^* = \frac{\varrho_R p_{T,L} - \varrho_L p_{T,R} + \varrho_L \varrho_R (v_{x,R} - v_{x,L})}{\varrho_R - \varrho_L}$. and $p_{T,K} = p_K + \frac{\mathbf{B}_K^2}{8\pi}$, the outer star states are given by

$$\rho_K^* = \frac{\varrho_K}{S_K - S_M} \tag{1.38a}$$

$$(v_x)_K^* = S_M \tag{1.38b}$$

$$(v_y)_K^* = v_y - \frac{B_{y,K}}{4\pi}\left[\frac{B_x(S_M - v_{x,K})}{\varrho_K(S_K - S_M) - \frac{B_x^2}{8\pi}}\right] \tag{1.38c}$$

$$(B_x)_K^* = B_x \tag{1.38d}$$

$$(B_y)_K^* = B_{y,K}\left[\frac{\varrho_K(S_K - v_{x,K}) - \frac{B_x^2}{8\pi}}{\varrho_K(S_K - S_M) - \frac{B_x^2}{8\pi}}\right] \tag{1.38e}$$

$$E_K^* = E_K\left(\frac{S_K - v_{x,K}}{S_K - S_M}\right) + \frac{p_T^* S_M - p_{T,K} v_{x,K}}{S_K - S_M} + \frac{Bx}{4\pi}\frac{(\mathbf{v}_K \cdot \mathbf{B}_K - \mathbf{v}_K^* \cdot \mathbf{B}_K^*)}{S_K - S_M} \tag{1.38f}$$

and similarly for the z components. If the denominators are zero (or sufficiently close to zero), then the transverse components of $\mathbf{v}$ and $\mathbf{B}$ are taken directly from w_K without rescaling.

Sometimes HLLD will compute a star state with nonpositive thermal pressure. If `HLLD_PHYSICAL_SAFETY` is enabled in `Config.h`, then DRAGON will compute the thermal pressure for the star states and switch to HLLE for a nonphysical result.

If the zero line falls between an alfven speed S_K^* and the corresponding outer speed, S_K, then the flux is given by

$$F_K^* = F_K + S_K(U_K^* - U_K) \tag{1.39}$$

on the appropriate side.

Inner Star States

The inner star states are

$$\rho_K^{**} = \rho_K^* \tag{1.40a}$$

$$(v_x)_K^{**} = S_M \tag{1.40b}$$

$$(v_y)_K^{**} = \frac{\sqrt{\rho_L^*}v_{y,L}^* + \sqrt{\rho_L^*}v_{y,R}^*}{\sqrt{\rho_L^*} + \sqrt{\rho_R^*}} \pm \frac{B_{y,R} - B_{y,L}}{\sqrt{4\pi\rho_L^*} + \sqrt{4\pi\rho_L^*}} \tag{1.40c}$$

$$(B_x)_K^{**} = B_x \tag{1.40d}$$

$$(B_y)_K^{**} = \frac{\sqrt{\rho_L^*}B_{y,R}^* + \sqrt{\rho_L^*}B_{y,L}^*}{\sqrt{\rho_L^*} + \sqrt{\rho_R^*}} \pm \frac{v_{y,R} - v_{y,L}}{\sqrt{\frac{1}{4\pi\rho_L^*}} + \sqrt{\frac{1}{4\pi\rho_R^*}}} \tag{1.40e}$$

$$E_K^{**} = \mp(\mathbf{v}_K^* \cdot \mathbf{B}_K^* - \mathbf{v}_K^{**} \cdot \mathbf{B}_K^{**})\sqrt{\frac{\rho_K^*}{4\pi}} \tag{1.40f}$$

and similarly for the z components. For the transverse components, the + path is taken if $B_x > 0$ and the $-$ path is taken if $B_x < 0$. For Energy, the minus path is taken upstream of B_x and the plus path is taken downstream of B_x (i.e. use the opposite sign to $B_x S_M$).

As with the outer star states, if `HLLD_PHYSICAL_SAFETY`, is enabled in `Config.h`, then DRAGON will revert to HLLE if the inner star state produces a nonphysical thermal pressure.

If the zero line falls between an alfven speed S_K^* and contact speed S_M, the flux is given by

$$F_K^{**} = F_K^* + S_K^*(U_K^{**} - U_K^*) \tag{1.41}$$

on the appropriate side.

Zero Normal B

When the interface-normal field vanishes, the Alfvén waves collapse onto the contact wave and the five-wave structure collapses to three regions. DRAGON handles this with a separate code path, using the same S_L, S_R, S_M as above but without the Alfvén-speed split.

Outer star states are given by

$$\rho_K^* = \frac{\varrho_K}{S_K - S_M} \tag{1.42a}$$

$$(v_x)_K^* = S_M \tag{1.42b}$$

$$(v_y)_K^* = v_y \tag{1.42c}$$

$$(\mathbf{B})_K^* = \mathbf{B}_K \left[\frac{S_K - v_{x,K}}{S_K - S_M}\right] \tag{1.42d}$$

$$E_K^* = E_K \left(\frac{S_K - v_{x,K}}{S_K - S_M}\right) + \frac{p_T^* S_M - p_{T,K} v_{x,K}}{S_K - S_M} \tag{1.42e}$$

No inner state is computed since $S_L^* = S_M = S_R^*$. In the exceptional case where $S_M = 0$ exactly, the flux is taken as the density-weighted average of the two one-sided fluxes $F = \frac{\sqrt{\rho_L^*}F_L^* + \sqrt{\rho_R^*}F_R^*}{\sqrt{\rho_L^*}+\sqrt{\rho_R^*}}$ to preserve left-right symmetry at that single point.

1.7.6 Roe (Hydrodynamics Only)

The Roe solver (Roe 1981; Roe & Pike 1984) linearizes the Euler flux Jacobian about the Roe-averaged state of Section 1.7.2, giving the five eigenvalues

$$\lambda = (v_{x,M} - a_M,\ v_{x,M},\ v_{x,M},\ v_{x,M},\ v_{x,M} + a_M) \tag{1.43}$$

which correspond to the left acoustic, entropy, two shear, and right acoustic modes.

The corresponding right eigenvectors $K_1, \ldots, K_5$ and wave strengths are

$$\alpha_1 K_1 = \frac{(p_R - p_L) - \rho_M a_M (v_{x,R} - v_{x,L})}{2a_M^2} \begin{pmatrix} 1 \\ \mathbf{v}_M - (a_M, 0, 0)^T \\ H_M - v_{x,M} a_M \end{pmatrix} \tag{1.44a}$$

$$\alpha_2 K_2 = (\rho_R - \rho_L) - \frac{p_R - p_L}{a_M^2} \begin{pmatrix} 1 \\ \mathbf{v}_M \\ \frac{1}{2}\mathbf{v}_M^2 \end{pmatrix} \tag{1.44b}$$

$$\alpha_3 K_3 = \rho_M (v_{y,R} - v_{y,L}) \begin{pmatrix} 0 \\ (0,1,0)^T \\ v_{y,M} \end{pmatrix}, \qquad \alpha_4 K_4 = \rho_M (v_{z,R} - v_{z,L}) \begin{pmatrix} 0 \\ (0,0,1)^T \\ v_{z,M} \end{pmatrix} \tag{1.44c}$$

$$\alpha_5 = \frac{(p_R - p_L) + \rho_M a_M (v_{x,R} - v_{x,L})}{2a_M^2} \begin{pmatrix} 1 \\ \mathbf{v}_M + (a_M, 0, 0)^T \\ H_M + v_{x,M} a_M \end{pmatrix} \tag{1.44d}$$

The flux is then computed by

$$F_{\text{Roe}} = \frac{1}{2}(F_L + F_R) - \frac{1}{2}\sum_{k=1}^{5} \alpha_k \left|\lambda_k\right| K_k. \tag{1.45}$$

However, the Roe method requires an entropy fix. (Harten & Hyman 1983). This is enabled by `Harten_Hyman` in `Config.h`. The entropy fix checks whether the left or right acoustic wave is a transonic rarefaction. If this is the case, the offending eigenvalue is replaced by an interpolated value between the fan's bounding speeds before the flux is assembled.

1.7.7 Riemann Solver Fallback Behaviour

Most Riemann solvers are not guaranteed to produce physically sound results. DRAGON provides mechanisms to adaptively pivoting to a different solver should one fail. One such mechanism is the `HLLD_PHYSICAL_SAFETY` pathway described in Section 1.7.5, which causes the HLLD solver to switch to HLLE in the event that a star state pressure is nonphsyical.

The other pathway is enabled via `RIEMANN_VERIFY_FALLBACK` in `Config.h`. In this method, the states $w_L - \xi\frac{\Delta t}{\Delta x}F_{i+\frac{1}{2}}$ and $w_R + \xi\frac{\Delta t}{\Delta x}F_{i+\frac{1}{2}}$ are calculated for $\xi =$`CONFIG::Riemann_ExactFallback_Parameter`. If either of the resulting quantities is nonphysical, the following fallback procedure is used

1. Recalculate the flux using HLLE (see Section 1.7.3)
   (a) If this result passes the above physicality test, return it
   (b) If this result fails the above physicality test, proceed to step 2
   (c) This step can be skipped by disabling `RIEMANN_FALLBACK_TRY_HLLE` in `Config.h`.
2. In Hydrodynamic mode, recalculate the flux using the Exact solver (see Section 1.7.1)
   (a) If this result passes the above physicality test, return it
   (b) If this result fails the above physicality test, proceed to step 3
   (c) In MHD mode, proceed directly to step 3.
3. Throw an exception, which will typically result in the entire timestep being restarted.

From our own testing we have found that `HLLD_PHYSICAL_SAFETY` is a more robust fallback pattern for the HLLD solver, and as such recommend disabling `RIEMANN_VERIFY_FALLBACK` for MHD builds.

1.8 Constrained Transport

DRAGON uses constrained transport to ensure that magnetic fields remain free of divergences. The specific implementation is largely based on Gardiner & Stone (2008).

1.8.1 Computation of Electric Fields

Because the induction equation is part of the same conservative MHD system solved by the Riemann solvers of Section 1.7, the resulting fluxes already contain the electric field (up to a sign) via the transverse-B components. Specifically, the flux of B^a through an b-face is $\mp E^c$, where the minus (plus) sign is taken when abc a cyclic (anticyclic) permutation of xyz. DRAGON extracts this information directly via `CT::computeElectric`, rather than reconstructing $\mathbf{E}$ separately.

In 3D, each edge-centered component of $\mathbf{E}$ touches four adjacent faces, and is taken as their average:

$$E^x_{i,j-\frac{1}{2},k-\frac{1}{2}} = \frac{1}{4}\left[F^{Z,B_y}_{i,j-1,k-\frac{1}{2}} + F^{Z,B_y}_{i,j,k-\frac{1}{2}} - F^{Y,B_z}_{i,j-\frac{1}{2},k-1} - F^{Y,B_z}_{i,j-\frac{1}{2},k}\right] \tag{1.46a}$$

$$E^y_{i-\frac{1}{2},j,k-\frac{1}{2}} = \frac{1}{4}\left[F^{X,B_z}_{i-\frac{1}{2},j,k-1} + F^{X,B_z}_{i-\frac{1}{2},j,k} - F^{Z,B_x}_{i-1,j,k-\frac{1}{2}} - F^{Z,B_x}_{i,j,k-\frac{1}{2}}\right] \tag{1.46b}$$

$$E^z_{i-\frac{1}{2},j-\frac{1}{2},k} = \frac{1}{4}\left[F^{Y,B_x}_{i-1,j-\frac{1}{2},j,k} + F^{Y,B_x}_{i,j-\frac{1}{2},k} - F^{X,B_y}_{i-\frac{1}{2},j-1,k} - F^{X,B_y}_{i-\frac{1}{2},j,k}\right] \tag{1.46c}$$

where $F^{X,B_y}_{i-\frac{1}{2},j,k}$ denotes the B^y component of the flux returned by the x-direction Riemann solve at face $(i-1/2, j, k)$, and similarly for the other flux/component pairs.

In 2D, the E_z component is likewise derived as

$$E^z_{i-\frac{1}{2},j-\frac{1}{2}} = \frac{1}{4}\left[F^{Y,B_x}_{i-1,j-\frac{1}{2},j} + F^{Y,B_x}_{i,j-\frac{1}{2}} - F^{X,B_y}_{i-\frac{1}{2},j-1} - F^{X,B_y}_{i-\frac{1}{2},j}\right]. \tag{1.47}$$

Meanwhile, the E_x and E_y components only touch one face:

$$E^x_{i,j-\frac{1}{2}} = -F^{Y,B_z}_{i,j-\frac{1}{2}}, \qquad E^y_{i-\frac{1}{2},j} = F^{X,B_z}_{i-\frac{1}{2},j} \tag{1.48}$$

CTU Upwinding

The above four-face average is only first-order accurate at the edge and can allow spurious checkerboard-like noise to grow. DRAGON corrects this in `CT::upwindElectric` using the unsplit Corner Transport Upwind (CTU) scheme of Gardiner & Stone (2008).

Each edge-centred $\mathbf{E}$ component is upwinded once per adjacent face, using the sign of the mass flux through that face to choose which side to draw from:

$$\text{upwindCorr}(\dot{m}, E^{\text{face}}_-, E^{\text{face}}_+, E^{\text{body}}_-, E^{\text{body}}_+) = \begin{cases} \frac{1}{4}\left(E^{\text{face}}_- - E^{\text{body}}_-\right) & \dot{m} > \epsilon, \\ \frac{1}{4}\left(E^{\text{face}}_+ - E^{\text{body}}_+\right) & \dot{m} < -\epsilon, \\ \frac{1}{8}\left(E^{\text{face}}_- + E^{\text{face}}_+ - E^{\text{body}}_- - E^{\text{body}}_+\right) & |\dot{m}| \le \epsilon, \end{cases} \tag{1.49}$$

with $\epsilon = 10^{-18}$ (`CTU_TOL` in `Electric.cpp`) guarding the near-zero-velocity case. Here $\dot{m}$ is the mass flux[7] through the relevant face. $E^{\text{body}}_\pm$ are the cell-centred reference values $\mathbf{E} = -\mathbf{v}\times\mathbf{B}$ (calculated by `CT::bodyElectric`) in the two cells adjacent to that face, and $E^{\text{face}}_\pm$ are the face-centred field values values (as used in `CT::computeElectric`) in the faces which 1) border both the cell and edge in question and 2) are not the face through which mass flux is taken. Each edge component receives one such correction per bordering face, and the corrections are summed onto the flux-averaged value from `CT::computeElectric`.

1.8.2 Faraday's Law

Given the finalized upwinded edge $\mathbf{E}$, the face-normal B-fields are advanced using

$$\frac{\partial \mathbf{B}}{\partial t} = -\nabla \times \mathbf{E}. \tag{1.50}$$

[7] The GS08 paper uses velocity, but mass flux will almost always have the same sign and fluxes are already being passed to `CT::upwindElectric`

`CT::Faraday` evaluates this curl such that each face component only involves the edge E components that actually bound it. In 3D, this is

$$\Delta B^x_{i-\frac{1}{2},j,k} = \frac{\Delta t}{\Delta z}\left[E^y_{i-\frac{1}{2},j,k+\frac{1}{2}} - E^y_{i-\frac{1}{2},j,k-\frac{1}{2}}\right] - \frac{\Delta t}{\Delta y}\left[E^z_{i-\frac{1}{2},j+\frac{1}{2},k} - E^z_{i-\frac{1}{2},j-\frac{1}{2},k}\right] \tag{1.51a}$$

$$\Delta B^y_{i,j-\frac{1}{2},k} = \frac{\Delta t}{\Delta x}\left[E^z_{i+\frac{1}{2},j-\frac{1}{2},k} - E^z_{i-\frac{1}{2},j-\frac{1}{2},k}\right] - \frac{\Delta t}{\Delta z}\left[E^x_{i,j-\frac{1}{2},k+\frac{1}{2}} - E^x_{i,j-\frac{1}{2},k-\frac{1}{2}}\right] \tag{1.51b}$$

$$\Delta B^z_{i,j,k-\frac{1}{2}} = \frac{\Delta t}{\Delta y}\left[E^x_{i,j+\frac{1}{2},k-\frac{1}{2}} - E^x_{i,j-\frac{1}{2},k-\frac{1}{2}}\right] - \frac{\Delta t}{\Delta x}\left[E^y_{i+\frac{1}{2},j,k-\frac{1}{2}} - E^y_{i-\frac{1}{2},j,k-\frac{1}{2}}\right] \tag{1.51c}$$

The corresponding 2D update follows the same pattern for the two in-plane face fields B^x, B^y (using E^z only) and the cell-centered B^z (using both E^x and E^y). This update, by construction, leaves $\nabla \cdot \mathbf{B}$ unchanged from its value before the step.

1.8.3 Calculation of Body-Centred Fields

Once the face-centred normal fields are calculated, the body-centred fields are calculated as follows

$$B^x_{i,j,k} = \frac{1}{2}\left(B^x_{i-\frac{1}{2},j,k} + B^x_{i+\frac{1}{2},j,k}\right) \tag{1.52a}$$

$$B^y_{i,j,k} = \frac{1}{2}\left(B^y_{i,j-\frac{1}{2},k} + B^y_{i,j+\frac{1}{2},k}\right) \tag{1.52b}$$

$$B^z_{i,j,k} = \frac{1}{2}\left(B^z_{i,j,k-\frac{1}{2}} + B^z_{i,j,k+\frac{1}{2}}\right) \tag{1.52c}$$

However, a complication arises from the magnetic contribution to energy. Unless the updated value happens to match the previous value exactly, it isn't possible to keep both total and thermal energy constant through the CT-update. DRAGON supports three different procedures for handling the energy in the CT update:

- *Always keep total energy constant* (`CT_CONSV_TOTAL_E`). This is done by converting $\mathbf{w}$ to conservative form, setting $\mathbf{B}$ on the conservative form, and then converting back to primitive form. This method ensures energy conservation, but may be prone to numerical issues where thermal pressure irrecoverably becomes nonphysical.
- *Always keep thermal pressure constant* (`CT_CONSV_THERMAL`). This is done by simply setting $\mathbf{B}$ on the primitive state $\mathbf{w}$. This method is the most robust, but will almost always violate energy conservation. This may be justifiable in some astrophysical flows on grounds that radiation violates energy conservation anyway.
- *Dynamically between the above options* (`CT_CONSV_BETA_GATED`). In this procedure, the local value of the plasma beta $\beta_m = \frac{8\pi p}{\mathbf{B}^2}$ is calculated and compared to `CONFIG::CT_Energy_Beta`. For cells with a plasma beta below the threshold, thermal energy is conserved to avoid numerical issues; otherwise, total energy is conserved. This option allows the simulation to remain closer to total energy conservation while still avoiding numerical issues.

The desired option can be selected by setting `CT_ENERGY_CONSV` in `Config.h`.

Chapter 2

Code Structure

2.1 Directory Layout

```
DRAGON/                    # repository root
|-- DRAGON/                # core solver source
|   |-- Boundary/          # boundary conditions
|   |-- FluidElement/      # primitive/conservative state & arithmetic
|   |-- Hydro/             # hydrodynamic solver
|   |   |-- CFL/           # timestep control
|   |   |-- Godunov/       # split/unsplit driver, flux application
|   |   |-- Reconstruction/# MUSCL interface-state reconstruction
|   |   |-- Riemann/       # Exact, HLL, HLLE, HLLC, HLLD, Roe solvers
|   |-- MHD/               # constrained transport (E-field, Faraday, body fields)
|   |-- Refinement/        # domain decomposition (DistGrid)
|   |-- main/              # program entry point & problem setup
|   |-- Config.h, Constants.h, Problem.cpp
|-- DRAGONWING/            # memory management & multithreading
|   |-- Memory_Management/
|   |-- Multithreading/
|-- DRAGONHOARD/           # HDF5 output & restart I/O
|-- DRAGONGAZE/            # Python visualisation/analysis scripts
|-- DRAGONLORE/            # this documentation
|-- Examples/              # example problem setups
|   |-- 1D/                # shock tubes
|   |-- Hydro/             # 2D and 3D Hydro examples
|   |   |-- Blast/, Diagonal/, Kelvin-Helmholtz/
|   |-- MHD/               # 2D and 3D MHD examples
|   |   |-- Blast/, LoopAdvection/, MHDRotor/, Orszag-Tang/, Waves/
|-- Testing/               # unit tests
|   |-- Core/
|-- LICENSE                #Apache License
|-- README.md
```

Listing 2.1: Repository layout

2.2 Data Structures

2.2.1 Fluid Elements

The following data types are defined in `DRAGON/FluidElement/FluidElement.h`

vec3

`DRAGON::vec3` is a plain structure with x, y, z components. The following operations are supported:

- Addition (`+`, `+=`) and Subtraction (`-`, `-=`) with another `vec3`.
- Multiplication (`*`, `*=`) and Division (`/`, `/=`) by a scalar `double`.
- Dot (`*`) and cross (`cross(v1, v2)`) product between two `vec3`'s.
- Swapping two components, either in place (e.g. `swapXY()`) or on a copy (e.g. `swappedXY()`).

PrimitiveState

`DRAGON::PrimitiveState` represents a single fluid element (i.e. one cell's state), which contains density `rho`, pressure `p`, velocity vector `v`, and (MHD only) magnetic field vector `B`. This is the form the solver reconstructs, applies boundary conditions to, and reports as output.

As already noted in Chapter 1, `B` is stored in Gaussian (non-rationalized) units, so the code is explicit about factors of 4π wherever they appear – concretely, `Constants.h` predefines `_1_4pi` ($= 1/4\pi$), `_1_8pi` ($= 1/8\pi$), and `sq4pi` ($= \sqrt{4\pi}$) rather than dividing by 4π inline.

ConservativeState

`DRAGON::ConservativeState` represents a fluid element in conservative form: mass density `rho`, energy density `E`, momentum density vector `mom`, and (MHD only) `B`. This is the form fluxes are computed and applied in: rather than a true fluid element, a particular instance of `ConservativeState` might instead represent a flux (times one computational unit time per computational unit length).

The following fluid arithmetic operations are supported by `ConservativeState`

- Addition (`+`, `+=`) and Subtraction (`-`, `-=`) with another `ConservativeState`.
- Multiplication (`*`, `*=`) and Division (`/`, `/=`) of all components by a `double`.
- Swapping two dimensions as with `vec3`. This swaps the corresponding components of both `mom` and `B`. `PrimitiveState` also supports this operation directly (using `v` instead of `mom`).

Both `PrimitiveState` and `ConservativeState` are constructible from the other (`ConservativeState(PrimitiveState)` and `PrimitiveState(ConservativeState)`). This can be done explicitly via the other, and is also done implicitly whenever a `PrimitiveState` is used in fluid arithmetic. If you need to directly add or subtract the components of two `PrimitiveState`'s, you will need to do this manually on each component.

Both types also provide a number of other physically relevant quantities, such as various wave speeds, `energy()` or `pressure()` on a primitive or conservative state, and `flux()` to calculate the fluxes of the governing MHD equations (1.1). The `flux()` function returns the flux in the x-direction, and in MHD includes the electric fields from the induction equation as the `B` components (see Section 1.8). When called from a `ConservativeState`, `flux()` can optionally take the velocity `v` if already known; otherwise `v` is computed from `mom` and `rho`.

2.2.2 ExtendedArray

Grid storage is built on a small family of templates in `DRAGON/Hydro/ExtendedArray`: `ExtendedArray1D<T>`, `ExtendedArray2D<T>`, and `ExtendedArray3D<T>`. Each owns a single flat, row-major heap array sized to include a configurable ghost margin. Accessing elements within an extended array is done via `array[i]`, `array[i,j]`, or `array[i,j,k]` depending on the array's dimension[1]. `i`, `j`, or `k` may be negative or exceed that dimension's size by an amount up to the array's ghost margin; in this case the accessed quantity corresponds not to a physical item but to cell within the boundary layer[2]. When DRAGON is built for testing, every cell access will assert that the bounds of the ghosts are within the valid range; this check is compiled out in production builds.

`ExtendedArray` objects can't be copied using normal C++ syntax, but the contents of one array can be copied to another (with identical dimension) using `clone(ref, copyghosts)`. Ghost cells are copied only if `copyghosts` is true.

`ArrayTypes.hpp` defines the concrete array types used throughout the solver as typedefs over these templates and `PrimitiveState`/`ConservativeState`/`vec3` (Section 2.2.1) – e.g. `FluidArray2D = ExtendedArray2D<PrimitiveState>`, `FluxArray3D = ExtendedArray3D<ConservativeState>`, `MagneticArray2D = ExtendedArray2D<vec3>`.

2.2.3 Grid

Fundamentally, a `DRAGON::Grid` (`DRAGON/Hydro/Grid.hpp`) is one or more extended arrays together with a boundary condition and the ability to advance the arrays in time.

The abstract `Grid` base class declares the two pure-virtual per-step advancement functions `split_step(dt)`/`unsplit_step(dt)`, the CFL-aware driver `advance(dt, check_cfl)` which wraps either `advance_split`/`advance_unsplit` (Section 1.5), and the virtual failure hook `on_step_fail(e)` used by the retry-with-halving loop.

The dimension-specific subclasses `Grid1D`/`Grid2D`/`Grid3D` each own one `FluidArray1D/2D/3D w` (Section 2.2.2) as their primary state, plus a staggered `MagneticArray2D/3D B` in MHD builds (Section 1.8). The magnetic field is stored on a staggered face-centred grid, with `B[i,j,k].x` corresponding to $B^x_{i-\frac{1}{2},j,k}$, `B[i,j,k].y` to $B^y_{i,j-\frac{1}{2},k}$, and `B[i,j,k].z` to $B^z_{i,j,k-\frac{1}{2}}$. External functions can access a reference to the staggered magnetic array via `grid._B()`.

2.3 Dependencies

DRAGON's only dependency is **HDF5**, used by DRAGONHOARD (Chapter 4)[3] for output and restart I/O. No other third-party library is required to build or run DRAGON itself.

DRAGON's in-house visualisation toolkit DRAGONGAZE (Chapter 5) depends separately on **h5py** (to read the HDF5 files DRAGONHOARD writes) and **Matplotlib** (to render plots from that data). These are Python-side dependencies only and do not affect the C++ build.

[1] It is for this reason that DRAGON requires C++23

[2] Or, in the case of domain decomposition (Section 3.3), it might be a cell owned by another subgrid.

[3] Core DRAGON and DRAGONWING are agnostic to the file storage format, so if HDF5 is for some reason unavailable on your system, you would only need to modify DRAGONHOARD to interface with your alternative format.

Chapter 3

Performance: DRAGONWING

DRAGON Wide Infrastructure Numerical Grids (DRAGONWING) is DRAGON's support library for memory management and multithreading.

3.1 Memory Management

DRAGON requires scratch arrays to store the information computed in the Reconstruction (Section 1.6), flux calculation (Section 1.7), and constrained transport (Section 1.8) stages. These scratch arrays are of the same size as the main grid, though the CT scratch arrays only hold 3-vectors instead of the full 8-component fluid states. Rather than have every call site `new`/`delete` these buffers, DRAGONWING centralises allocation behind three request functions, overloaded for 1D/2D/3D grids:

- `requestPrimitiveArrays(N, ...)` – returns `N` scratch arrays of `PrimitiveState`
- `requestFluxArrays(N, ...)` – returns `N` scratch arrays of `ConservativeState` (used to store fluxes)
- `requestVec3Arrays(N, ...)` – returns `N` scratch arrays of `vec3` (used e.g. for electric fields).

Each of these returns a `DRAGONWING::ArrayGuard`, a small RAII wrapper templated on the underlying `ExtendedArray` type. `ArrayGuard` provides a collection of raw pointers to the allocated arrays, which are returned to DRAGONWING upon destruction of the `ArrayGuard`. Arrays reclamation happens automatically when the guard goes out of scope (exception-safely included), or can be done early by calling `guard.release()`. Note that it is the responsibility of the caller to cease all use of any arrays allocated by the guard once `release()` is called.

`REUSE_AUX_GRIDS` is enabled (this is the default) in `DRAGONWING_Config.hpp`, DRAGONWING keeps a static pool of previously-allocated arrays for each combination of element type (`PrimitiveState`/`ConservativeState`/`vec3`) and dimensionality (1D/2D/3D). Each of the three element types has its own `std::mutex` (`pm`, `cm`, `vm`), so pooling is thread-safe. Each request for new arrays first scans the corresponding pool for inactive arrays whose size match the request. Any such arrays are then marked as active and reused as is. Only if the existing arrays are insufficient to meet the caller's request are new arrays allocated with `new`; these arrays are then added to the pool and marked as active. When an array is released (via `ArrayGuard::release()` it is marked inactive again; the memory is not freed and is instead available for reuse. Because pooled arrays can carry stale data from a previous use, any code that requests scratch arrays must treat their contents as unspecified before writing to them. DRAGONWING makes no guarantee of zero-initialization. Arrays are only deleted when the program terminates, or if `DRAGONWING::purgeAllBuffers()` is called. This function

frees every array currently sitting inactive in the pools via `delete`, though Active arrays are left alone so as to not interfere with the currently executing process; these arrays will instead be freed individually the next time they are released back to DRAGONWING.

If instead `REUSE_AUX_GRIDS` is disabled, Requests for arrays always allocate arrays with `new`, and releasing arrays always frees the array with `delete`. No pool is kept, so there is nothing to purge and no stale-data concern, at the cost of paying full allocation/deallocation overhead on every step.

3.2 Multithreading

`DRAGONWING::ThreadPool` is used by `DistGrid` (see Section 3.3) to advance sibling subgrids concurrently, each on its own thread. It is constructed with a fixed thread budget `N` (one thread per child subgrid); `launchParallel(grid, dt)` is then called once per child to start `grid->advance(dt, false)`[1] running on a new thread.

3.2.1 Checkpoint Protocol

From a synchronisation standpoint, each-step advancement of a subgrid is divided into two phases:

1. Each subgrid, computes all updated values and verifies that the solution is physically admissible (steps 1-6 in Section 1.4.2). The original array remains in tact so that the step can be restarted if necessary. In an unsplit update, this phase uses a significant amount of scratch memory.

2. Each subgrid commits the preliminary update to the base array (step 8 in Section 1.4.2).

All subgrids must complete phase 1 before any can enter phase 2. This requirement allows any subgrid to request that the step be restarted before the existing data is overwritten.

Subgrids maintain these synchronisation phases as follows:

1. `DRAGONWING::waitForRelease()` (Optional but recommended). Since phase 1 typically uses a significant amount of scratch memory, the user might optionally set `DRAGONWING::CONFIG::phase_1_max_threads` in `DRAGONWING_Config.hpp` to limit program's total memory usage. The `ThreadPool` will track the number of threads currently in phase 1. If the number of threads is already at its limit, the caller will be blocked until another thread either reports completion of phase 1 or requests a step restart. `waitForRelease()` returns a boolean, which is true if the caller is clear to proceed with phase 1 and false if another subgrid has requested a restart (in which case the calling function should return instead of proceeding).

2. Do the work of phase 1.

   (a) If an error occurs, call `DRAGONWING::requestRestart(error_msg)` to request that all subgrids restart the current step. For the first subgrid to request a step restart, the `error_msg` string can be accessed via `ThreadPool.restartMsg()`. Calling `restartMsg()` will then clear the message.

3. `DRAGONWING::reportChckpoint1()`. This informs the local `ThreadPool` that another subgrid has completed phase 1. This will wake up any threads currenlty blocked in `waitForRelease()` and will allow one to proceed, or if it is the last function to complete it will wake up all threads blocked by in the next step

[1] `grid->advance(dt, false)` bypasses the CFL checks of Section 1.5 since these are handled by the parent.

4. `DRAGONWING::waitForCheckpoint1()`. This will block the current thread until all subgrids have completed phase 1. This function returns a boolean, which is true if the caller is clear to proceed with phase 2 and false if another subgrid has requested a restart (in which case the calling function should return instead of proceeding).

5. Do the work of phase 2

6. `DRAGONWING::reportChekcpoint2()`. This informs the thread pool that this subgrid has completed its work.

The implementations of these functions lives in the `ThreadPool` class, but the caller need not know which `ThreadPool` is in use. DRAGONWING maintains a `thread_local` variable, allowing the `DRAGONWING ::` functions to call the corresponding function on the `ThreadPool` that created the thread from which it is called.

After all subgrids are launched via `launchParallel(grid, dt)`, call `waitForCompletion()` on the same `ThreadPool` object. This will wait until either either all threads have reported completion of phase 2 or any thread has requested a step restart. `waitForCompletion()` will return true if the step was successful, or false if it needs to be restarted.

3.3 Domain Decomposition

`DRAGON::DistGrid<T>` implements domain decomposition, in which a larger gird is split into smaller subgrids which can be advanced concurrently on separate threads. Although this class is found in `DRAGON/Refinement`[2] instead of `DRAGONWING/` proper, it is documented in this chapter because it is DRAGON's only current form of parallelism and depends directly on the `ThreadPool`/checkpoint machinery of Section 3.2.

`DistGrid<T>` is a CRTP-style base combined with `Grid1D`/`Grid2D`/`Grid3D` by multiple inheritance to form `DistGrid1D`, `DistGrid2D`, and `DistGrid3D`. Each holds a `std::vector<std::unique_ptr<T>>` of children – subgrids of the same concrete type. The tree can in principle nest to arbitrary depth, but until such time that AMR is implemented DRAGON will in practice only ever build to one level: a parent (constructed with `root=true`) and its children (constructed with `root=false`). A grid with `children.size() <= 1` is treated as a leaf and steps itself directly rather than delegating to its (nonexistent or single) children.

3.3.1 Load Balancing

The number of children along each axis is chosen to keep child subgrids close to square (2D) or cubic (3D), so that surface-to-volume overhead from ghost-cell synchronization is not concentrated in one direction:

- **1D:** `ncx = CONFIG::core_count` directly – every core gets one child.
- **2D:** solving $n_x/n_{cx} = n_y/n_{cy}$ and $n_{cx}n_{cy} \gtrsim$ core_count gives

$$n_{cx} = \left\lceil \sqrt{\text{core\_count} \cdot \frac{n_x}{n_y}} \right\rceil, \qquad n_{cy} = \left\lceil \sqrt{\text{core\_count} \cdot \frac{n_y}{n_x}} \right\rceil. \tag{3.1}$$

[2]DRAGON does not yet support Adaptive Mesh Refinement (AMR), but the `Refinement` directory uses this name in anticipation of future AMR implementation since Domain Decomposition is required for AMR. When such implmentation is added, this section will likely move to the AMR chapter.

- **3D:** the analogous cube-root balancing is used for n_{cx}, n_{cy}, n_{cz}, giving

$$n_{cx} = \left\lceil \left(N \cdot \frac{n_x^2}{n_y n_z} \right)^{\frac{1}{3}} \right\rceil, \qquad n_{cy} = \left\lceil \left(N \cdot \frac{n_y^2}{n_x n_z} \right)^{\frac{1}{3}} \right\rceil, \qquad n_{cy} = \left\lceil \left(N \cdot \frac{n_z^2}{n_x n_y} \right)^{\frac{1}{3}} \right\rceil, \quad (3.2)$$

  where N is equal to `CONFIG::core_count`.

Because each factor is rounded up independently, the product $n_{cx} n_{cy}(n_{cz})$ is not guaranteed to equal `core_count` exactly – DRAGON may use somewhat more subgrids (and hence threads) than `core_count` when the aspect ratio does not divide evenly.

Given a child count n_c along an axis, `computeChildSize(nx, nc, i)` distributes the n_x cells as evenly as possible: every child gets `nx / nc` cells, and the first `nx % nc` children (by index) each get one extra cell, so no two children differ in size by more than one cell.

3.3.2 Ghost Cells

Child subgrids need enough ghost cells to reconstruct correctly at their boundaries without an intra-step resynchronization with their siblings. `validGhosts(g)` enforces a scheme-dependent minimum, taking the larger of the caller's requested depth `g` and the required minimum number of ghosts. In general, a grid needs the following number of ghosts

- If MUSCL reconstruction is disabled, only a single ghost (the boundary) is required.
- If MUSCL reconstruction is enabled, a second ghost is required in order to reconstruct the boundary state correctly.
- For unsplit (CTU) integration with MHD, a third ghost is requried. This is because constrained transport needs the first layer of transverse fluxes in the boundary, and with CTU integration these fluxes are effected transverse corrections from the third layer of ghosts from the boundary.

However, for split integration, twice as many ghosts are needed as a result of each step performing two half-steps along one or more dimensions.

Because a child subgrid's ghost cells are filled directly by its parent rather than through the standard boundary-condition API, every child is constructed with `boundary = Boundary::Ignore()` (Section 1.3).

3.3.3 Parallel Step & Synchronization

`DistGrid<T>::step(dt)` is the actual parallel execution, called whenever a `split_step`/`unsplit_step` is invoked on a grid that has more than one child:

1. `pushToChildren()` copies the parent's fluid state (including its own ghost layer) into each child, offset appropriately for that child's position in the domain, then recurses into `child->pushToChildren()` so any grandchildren would be synchronised too.
2. Construct a `DRAGONWING::ThreadPool` sized to the number of children, and call `launchParallel(child, dt)` for each one, launching its `split_step`/`unsplit_step` on its own thread.
3. `waitForCompletion()` blocks until every child has reported checkpoint 2 (Section 3.2) or a restart is requested.

4. If any child failed, throw a `std::runtime_error` carrying the pool's restart message – this propagates up into the same retry-with-halving substep loop as any other step failure (Section 1.5).

5. Otherwise, `loadFromChildren()` folds the children's updated interiors back into the parent. This function copies back only the child's interior cells (no ghosts) into the corresponding region of the parent. It also calls each child's `loadFromChildren()` prior to folding the child's data to ensure any grandchildren would be synchronised too.

Leaf grids (`children.size() <= 1`) bypass all of this: their `split_step`/`unsplit_step` simply calls the corresponding base `Grid1D/2D/3D` method directly and reports checkpoint 2 itself, since there are no children to launch or wait on. `on_step_fail()` follows the same split: a leaf delegates to the base `Grid`'s recovery logic, while a parent (with more than one child) always returns `true` – there is no grid further up the decomposition to hand restart responsibility to, so the parent itself must trigger the retry.

Chapter 4

File Output: DRAGONHOARD

DRAGON Hierarchical Output for Analysis, Restarts, and Diagnostics (DRAGONHOARD) is DRAGON's I/O library: it writes simulation output, and supports loading them back in as restart-checkpoints. The public interface (`DragonHoard.hpp`) exposes `writeToFile`/`loadFromFile`, each overloaded once per grid dimensionality plus a `Grid`-dispatching version that picks the right overload with a `dynamic_cast`.

Each frame is written as a single HDF5 file named `<output_base_name>_#####.h5`, where the five-digit (zero-padded) cycle number comes from `cycle_string()` and `output_base_name` is set in the module's configuration header `DRAGONHOARD_Config.h`. DRAGONHOARD automatically appends the `.h5` extension to a bare filename when it is not already present, so callers of `writeToFile`/`loadFromFile` do not need to include it themselves.

The output directory `output_dir` is also set in `DRAGONHOARD_Config.h`. `verifyOutputDirectory()` creates `output_dir` if it does not already exist (and throws if the path exists but is not a directory); it is expected to be called once before the first write of a run (the default DRAGON `main.cpp` does this).

4.1 File Format

Every array is written gzip-compressed at level `HDF5_COMPRESSION_LEVEL` (set in `DRAGONHOARD_Config.h`, nonpositive to disable), chunked at up to 1024 cells (1D), 32×32 (2D), or $16 \times 16 \times 16$ (3D). Chunking is capped to the array's own extent on small grids.

Each file also carries a small set of scalar attributes describing its contents: `format_version`, `write_option`, `dim`, `MHD`, `nx`/`ny`/`nz`, `dx`/`dy`/`dz`, `cycle`, and `time`. `ng` contains the ghost depth actually written, though this is normally zero[1] since ghosts are normally regenerated by the boundary conditions (Section 1.3). 2D/3D MHD files additionally store `B_floats`, recording the precision of the recorded body-centred fields.

Density (`rho`) is always written as are face-normal B fields `Bf` (if MHD is enabled). The other components depend on `HDF5_WRITE_OPTION` and `HDF5_REDUNDANT_VALS_OPTION`.

Which variables are written is controlled by `HDF5_WRITE_OPTION` in `DRAGONHOARD_Config.h`. This bitmask is as follows:

- bit 1 – primitive variables (`v`, `p`)
- bit 2 – energy density (`E`)
- bit 4 – conserved variables (`mom`, `E`)

[1] If you do need write ghosts for debugging purposes, enable `WRITE_GHOSTS_TO_FILE` in `DRAGONHOARD_Config.h`

The following presets are given: `HDF5_WRITE_PRIMITIVE` (1), `HDF5_WRITE_PRIMITIVE_AND_ENERGY` (3, the default), `HDF5_WRITE_CONSERVATIVE` (6), and `HDF5_WRITE_PRIMITIVE_AND_CONSERVATIVE` (7, both forms, at the cost of a larger file). A `#error` in `HDF5Output.cpp` rejects any option that sets neither the primitive nor the conservative bit, since that would leave the fluid state under-determined.

Body-centred `B` in MHD builds is written whenever `HDF5_REDUNDANT_VALS_OPTION` is not `HDF5_WRITE_OMIT`; that same setting also controls whether it (and, when the energy bit is set without the momentum bit, `E`) is stored as `float` or `double`. These values can be recomputed by loader, so trading precision for file size is safe in this case.

4.2 Restart and Loading

Loading mirrors writing: `loadFromFile` dispatches to the matching `Grid1D`/`Grid2D`/`Grid3D` overload and opens the file read-only. Before touching the grid it checks compatibility, rejecting a file with a newer `format_version` than this build understands, a mismatched dimensionality or dimensions[2], or (in a hydro build) a file that was written with MHD enabled.

Density and (in MHD builds) magnetic fields are loaded independent of `write_option`. The remaining load is determined by the file's `write_option`, not the current build's `HDF5_WRITE_OPTION` setting. If the file contains the full primitive state, then velocity and pressure are read; only if the file is conservative-only are energy and momentum read and subsequently converted to primitive states. Finally, if the body-centred `B` was not stored at full precision (`B_floats` $\neq$ `HDF5_WRITE_DOUBLE`), the loader discards whatever it read and calls `grid.initialize_B_fields()` to recompute it exactly from the (always double-precision) face field.

Whether a run attempts to restart at all is controlled at compile time by `RESTART_FROM_FILE` in `DRAGONHOARD_Config.h`. When restarts are enabled, `RESTART_FRAME` selects which frame to load; if negative (the default), `restartFileName()` scans `output_dir` and restarts from whichever `..._#####.h5` file has the highest cycle number.

[2] `nx`/`ny`/`nz` must match exactly, there is currently no support for restarting onto a differently sized grid.

Chapter 5

Making Plots: DRAGONGAZE (Alpha)

DRAGON Graphical Analysis & Zonal Exploration (DRAGONGAZE) is DRAGON's Python plotting toolkit: a set of standalone scripts that turn the HDF5 frames written by DRAGON (Chapter 4) into PNG images, using Matplotlib. Nothing here needs to be compiled or linked against DRAGON – `DRAGONGAZE/` is a Python package, so each script is run as a module from the command line, from the directory that *contains* `DRAGONGAZE/`, once a simulation has produced at least one output frame.

5.1 Setup

DRAGONGAZE needs **h5py** and **Matplotlib** installed (Section 2.3). Before running any script, check two settings at the top of `Config.py`: `hdf_dir` must point to the same directory the simulation is writing output to (i.e. DRAGONHOARD's own `output_dir`, Section 4.1), and `img_dir` is where the generated plots will be saved. Be sure to create this directory first if it doesn't already exist.

5.2 Generating Plots

Every command below is run from the directory that contains `DRAGONGAZE/` (e.g. the DRAGON repository root), not from inside it – see Section 5.4 for why.

- **1D runs:** run `python -m DRAGONGAZE.GAZE1D` for hydrodynamics, or `python -m DRAGONGAZE.GAZE1DMHD` for MHD. No arguments are needed – each produces one multi-panel image per existing frame, combining density, pressure, energy, and velocity (plus, for MHD, the transverse magnetic field components) into a single figure.
- **2D/3D runs:** list the fields to plot on the command line, e.g. `python -m DRAGONGAZE.GAZE2D rho p Bmag` or `python -m DRAGONGAZE.GAZE3D rho Bmag`. Each named field gets its own image per frame. Available field names are `rho`, `vx`, `vy`, `vz`, `p`, `E`, and (MHD builds) `Bx`, `By`, `Bz`, plus `Bmag` for the field strength $|\mathbf{B}|$ – this is computed automatically, so there's no need to request the components separately just to see it.
- **3D slices:** a field can optionally be suffixed with an axis pair to pick which mid-plane slice to render, e.g. `rho-xy`, `p-xz`, `Bmag-yz` (order doesn't matter – `xy` and `yx` give the same slice). A field with no suffix produces all three mid-plane slices. A field can be listed twice to select two of the three possible slices, for example `rho-xy rho-xz`.

Images are written into `img_dir`, named by field, frame number, and (for 3D) axis pair (e.g. `density_frame_00003.png`, or `density_xy_frame_00003.png` for a 3D slice). A sequence of frames for one field sorts naturally and can be stitched directly into an animation (e.g. with `ffmpeg` or `magick`). To keep such an animation from flickering, DRAGONGAZE fixes each field's colour and axis limits once across every existing frame before it starts plotting, rather than rescaling them frame by frame.

5.3 Customizing Plot Appearance

Everything about how a plot looks is controlled from `Config.py` – the plotting scripts themselves never need to be touched:

- `titles` and `labels` set each field's plot title and axis label – labels support LaTeX-style math via Matplotlib's mathtext, so a density label can be written to render as ρ rather than plain text.
- `cmaps` sets the colormap used for each field; any colormap name from Matplotlib's reference can be used.
- `log_plots` switches a field between a linear and a logarithmic scale.
- `x_mode`/`y_mode`/`z_mode` choose where each axis's origin is labelled from – centred, or at one edge. This is purely cosmetic and never changes the underlying data.
- `plot_title` sets a title shared across every plot, and `time_unit` appends a unit label after the timestamp shown on each plot. DRAGON is unitless by default, so this is left blank unless a specific run has been given physical units.

5.4 Additional Information for Developers

`HDF5_keys.py` maps each plottable field name to the HDF5 dataset path DRAGONHOARD wrote it under (Section 4.1). These paths are hard-copied from `HDF5_Attrs.hpp` rather than shared with the C++ side in any way, so a key renamed or added there has no effect until `HDF5_keys.py` is updated to match. The failure mode is a plain `KeyError` at plot time, not a build error.

`FileUtils.readField(n, field)` is what actually resolves a field name to an array: for every field except `Bmag` this is a direct lookup via `HDF5_keys.keys`, while `Bmag` is synthesized on the fly as $\sqrt{B_x^2 + B_y^2 + B_z^2}$ since it isn't stored directly. You can use the same pattern to (checking for a special-cased name before falling back to the `keys` lookup) is the place to add any other derived quantity (such as kinetic energy). Be sure to also add entries for the your new field in `Config.py`.

Chapter 6

Installation and Usage

Chapter 7

Validation and Test Cases

Appendix A

Changelog

26 August 2026: Initial documentation for DRAGON v1.0